

Conception et Implémentation d'une Intelligence Artificielle pour le Jeu d'Escampe

Rapport Final de Projet

Mathieu WAHARTE

Raphaël BINDA

Mai 2026

Résumé

Ce rapport détaille la conception et la mise en œuvre d'un agent autonome capable de jouer au jeu d'Escampe dans un environnement avec contraintes de temps réel. Nous y présentons nos choix d'implantation, notamment les structures de données optimisées (Bitboards), les algorithmes de recherche (Negamax avec élagage Alpha-Bêta, Iterative Deepening) ainsi que la fonction d'évaluation heuristique développée pour modéliser la complexité topologique du plateau. Nous abordons également les stratégies de placement initial, la gestion dynamique du temps d'exécution, et fournissons une analyse des performances validées par des tests rigoureux.

Table des matières

1	Introduction	3
1.1	Fin de Partie	3
1.2	Analyse de la Complexité	3
2	Choix d'Implantation et Structures de Données	3
2.1	Représentation du Plateau (Bitboards)	4
2.2	Optimisations d'Implémentation	4
3	Algorithmes de Recherche du Meilleur Coup	4
3.1	Negamax et Élagage Alpha-Bêta	4
3.2	Approfondissement Itératif (Iterative Deepening)	5
3.3	Ordonnancement des Coups et Élagage Sélectif	5
4	Fonction d'Évaluation (Heuristiques)	6
4.1	Heuristiques Implémentées et Testées	6
4.2	Exploration d'une Évaluation par Réseau de Neurones : BandDPER	6
5	Stratégie Initiale : Placement et Ouvertures	10
5.1	Principes de Placement (Opening Book)	10
5.2	Stratégie par Phase de Jeu	11

6	Gestion du Temps Réel	11
7	Analyse des Performances et Tests	11
7.1	Évaluation Elo et Test SPRT	12
7.2	Validation de la Conduite Autonome	12
8	Conclusion et Difficultés Rencontrées	12

1 Introduction

Dans le cadre de ce projet, l'objectif principal est de développer une Intelligence Artificielle capable de conduire de façon autonome une partie d'Escampe en temps limité (300s à 900s par joueur). L'agent doit communiquer avec un arbitre validant les coups et garantissant le respect des contraintes temporelles. Le jeu d'Escampe se caractérise par des règles de déplacement atypiques (dépendance à la bande ciblée par l'adversaire) et une asymétrie de la topologie du plateau. Ces spécificités requièrent des choix algorithmiques pointus, s'écartant parfois des implémentations classiques des jeux à information parfaite.

1.1 Fin de Partie

Une partie se termine dès qu'un joueur capture la licorne adverse en y déplaçant un paladin. Aucun score partiel n'existe — seule la capture compte. Pour les agents, il convient d'ajouter un mécanisme de détection de répétition de position afin d'éviter les boucles infinies.

1.2 Analyse de la Complexité

Facteur de branchement. Chaque pièce peut se déplacer dans au plus 4 directions lors de son premier saut, puis 3 directions pour les suivants (sans revenir sur ses pas), avec jusqu'à 3 sauts selon sa bande. Dans le pire des cas, avec 6 pièces :

$$b_{\max} = 6 \times (4 + 2 \times 3) = 60 \text{ coups légaux par tour.}$$

En pratique, la contrainte de bande, les bords du plateau et les blocages ramènent ce facteur à 15–25 coups légaux en position typique de milieu de partie, ce qui justifie l'emploi d'un élagage Alpha-Bêta agressif.

Majorant du nombre d'états. En modélisant un état de jeu par les positions des 12 pièces parmi 36 cases et la bande imposée par le dernier coup (4 valeurs), on obtient :

$$\Updownarrow = \binom{36}{1} \binom{35}{5} \binom{30}{1} \binom{29}{5} \times 4 = \frac{36!}{(5!)^2 (36-12)!} \times 4 \approx 1,67 \times 10^{14}$$

états distincts. Ce chiffre confirme que toute exploration exhaustive est impossible et que les heuristiques et l'élagage sont indispensables.

Sources de difficulté. Au-delà de la complexité combinatoire, les principales difficultés algorithmiques sont : (i) la contrainte de bande qui conditionne les pièces jouables au tour suivant, créant des effets de tempo et de forçage ; (ii) l'absence de simplification rapide (pas de capture de pions), qui allonge les séquences tactiques ; (iii) l'importance du placement initial qui engendre une forte variabilité de départ.

2 Choix d'Implantation et Structures de Données

Afin de garantir des performances optimales lors de l'exploration de l'arbre de jeu, les structures de données sous-jacentes doivent permettre une génération de coups et une évaluation d'états extrêmement rapides.

2.1 Représentation du Plateau (Bitboards)

Nous avons opté pour une représentation du plateau basée sur des *Bitboards* [2]. Le plateau de 6x6 cases est encodé sur des entiers de 64 bits (`long` en Java).

- Les opérations de manipulation de pièces (génération des coups, détection de collisions) se réduisent à des opérations bit à bit (ET, OU, décalages), exécutées en un seul cycle d'horloge.
- Cette approche permet d'éliminer les itérations coûteuses sur des tableaux multidimensionnels et minimise la création d'objets, limitant ainsi la pression sur le *Garbage Collector*.

2.2 Optimisations d'Implémentation

Plusieurs choix techniques ont été faits pour maximiser le nombre de nœuds explorés par seconde :

- **Suppression des appels** `instanceof` lors de la génération des coups via des méthodes dédiées par type de pièce, évitant le coût du dispatch polymorphique à chaque nœud.
- **Mesure temporelle haute résolution** : utilisation de `System.nanoTime()` plutôt que `System.currentTimeMillis()` pour une résolution à la nanoseconde sans dépendance à l'horloge système.
- **Stratégie Make-Unmake** (annulation de coup) : plutôt que de cloner le plateau à chaque nœud de l'arbre, les coups sont joués sur le plateau courant puis défaits après évaluation (`board.undo(move, role)`). Cette approche élimine toute allocation mémoire dans la boucle de recherche et réduit la pression sur le *Garbage Collector*.
- **Pré-allocation des structures** : les tableaux de positions de pièces (paladins, licornes) et les masques de Bitboards sont alloués une seule fois. Le tri de la liste des coups est réalisé en place (*insertion sort* sans copie, cf. section 3.3).
- **Table de transposition (Zobrist Hashing)** [10] : le schéma d'encodage (XOR de valeurs aléatoires 64 bits par pièce et par case, entrées de table stockant profondeur, type de nœud, score et meilleur coup) a été entièrement conçu mais non intégré dans la version finale par contrainte de temps. Elle constitue l'optimisation la plus prometteuse pour des versions futures, pouvant notamment permettre la détection de répétition de position.

3 Algorithmes de Recherche du Meilleur Coup

La recherche du meilleur coup repose sur la théorie de la recherche adversarielle pour les jeux à somme nulle.

3.1 Negamax et Élagage Alpha-Bêta

Nous avons implémenté l'algorithme Negamax (une variante élégante du Minimax) couplé à un élagage Alpha-Bêta [1]. Cette approche unifie l'évaluation pour les deux joueurs, réduisant ainsi la complexité du code. L'élagage Alpha-Bêta permet de réduire

drastiquement le facteur de branchement effectif de l'arbre de recherche en ignorant les branches dont il est prouvé qu'elles sont sous-optimales.

3.2 Approfondissement Itératif (Iterative Deepening)

Pour exploiter au mieux le temps imparti, la recherche est structurée autour d'un approfondissement itératif [3]. L'algorithme explore l'arbre progressivement : à profondeur 1, puis 2, puis 3, et ainsi de suite jusqu'à la limite `depthMax`. À chaque nouvelle itération de profondeur, le temps écoulé est vérifié via notre limite de calcul logicielle (*Soft Bound*). Si cette limite est atteinte en cours d'itération, l'algorithme abandonne l'itération incomplète en cours et renvoie immédiatement le meilleur coup validé lors de la *dernière itération complétée*. Ce mécanisme garantit qu'un coup fiable et profondément évalué est toujours disponible sans risque de dépassement.

Une amélioration envisagée mais non intégrée dans les délais du projet est l'*Aspiration Windows* : au lieu de relancer chaque itération avec une fenêtre pleine ($\alpha = -\infty$, $\beta = +\infty$), on initialiserait α et β autour du score de la profondeur précédente (par exemple ± 50). En cas d'échec (*fail-low* ou *fail-high*), la fenêtre serait élargie et la recherche relancée. Cette technique peut réduire significativement le nombre de nœuds à explorer lorsque le score est stable entre profondeurs successives.

3.3 Ordonnancement des Coups et Élagage Sélectif

L'efficacité de l'élagage Alpha-Bêta dépend directement de la qualité de l'ordonnancement des coups [6] : dans le cas idéal, chaque nœud MIN ne considère qu'un seul coup (l'élagage est immédiat). Les deux techniques suivantes, encapsulées dans la classe `MoveOrderer`, ont été implémentées.

Killer Heuristic [7]. À chaque profondeur de l'arbre, on maintient un tableau de 2 *killer moves* — coups ayant provoqué une coupure bêta dans un nœud frère à la même profondeur. Ces coups sont essayés en priorité (score fixe `KILLER_SCORE = 900 000`), car il est probable qu'ils causent à nouveau une coupure dans le nœud courant. L'enregistrement suit une file FIFO à 2 slots : le nouveau killer déplace le premier, qui passe en second slot, préservant ainsi un *backup* fiable.

History Heuristic. Un tableau plat `historyTable[36 × 36]` indexé par l'encodage du coup (case de départ $\times$ 36 + case d'arrivée, soit 1 296 entrées sur un plateau 6×6) accumule un score à chaque coupure bêta, pondéré par profondeur². Ce poids favorise les coupures profondes, statistiquement plus significatives. Les coups historiquement les plus coupants remontent naturellement en tête de liste sans aucun coût d'allocation.

Tri in-place par insertion. L'ordonnancement combine les deux scores (killer prioritaire, puis historique) et trie la liste de coups par insertion directement dans l'objet `ArrayList` renvoyé par `possibleMoves()`. Le tri par insertion est optimal pour de petites listes — le nombre de coups légaux en Escampe ne dépasse généralement pas 30.

Techniques non implémentées. Le *Transposition Table Move* (essai en tête de liste du meilleur coup issu de la table de transposition [7]), le *Null-move Pruning* [8] et le *Late Move Reduction* [9] ont été identifiés comme des améliorations à fort potentiel, mais n'ont

pas été intégrés dans les délais du projet. Ils constituent des priorités pour des versions ultérieures.

4 Fonction d'Évaluation (Heuristiques)

L'évaluation des feuilles de l'arbre de recherche se doit d'être précise tout en demeurant très rapide à calculer. Notre heuristique est une combinaison linéaire de plusieurs signaux topologiques.

4.1 Heuristiques Implémentées et Testées

La fonction d'évaluation s'articule autour des métriques suivantes :

1. **Distances d'Attaque et de Défense** : Calcul de la distance de Manhattan entre les Paladins et les Licornes adverses. Plus les attaquants sont proches, meilleur est le score. L'heuristique pénalise la proximité des Paladins adverses à notre Licorne. *Note : une implémentation avancée utilisant une distance en nombre de sauts (BFS ply) intégrant la contrainte des bandes a également été modélisée.*
2. **Échappatoire de la Licorne (Escapability)** : Compte direct du nombre de cases légales de fuite pour la Licorne de chaque joueur. Un adversaire avec peu d'échappatoires est considéré comme dominé positionnellement.
3. **Contrôle des Bandes (Band Control)** : La contrainte du jeu force l'adversaire à partir d'une bande spécifique. Le positionnement de nos Paladins sur des cases de bande 1 (où l'adversaire n'a aucun choix) est fortement valorisé.
4. **Pression Temporelle et Mobilité** : Récompense accordée pour un nombre de coups légaux plus élevé, et forte pénalité lorsqu'un joueur est forcé de passer son tour en raison du mécanisme de contrainte.

Calibration des poids par SPSA. Les neuf poids de la formule ont été calibrés empiriquement via l'algorithme SPSA (*Simultaneous Perturbation Stochastic Approximation*), une méthode de descente de gradient stochastique particulièrement adaptée à l'optimisation de fonctions d'évaluation de jeux [11]. À chaque itération, les paramètres sont perturbés dans deux directions aléatoires ($\pm\delta$), deux parties sont jouées (une dans chaque configuration), et le gradient estimé est utilisé pour mettre à jour les poids. Le processus a été exécuté sur 50 itérations avec 16 paires de parties par itération (32 parties/itération), soit 1600 parties par run, en parallèle sur tous les cœurs disponibles.

Une étude d'ablation a révélé que le terme de contrôle des bandes ($w_7\mathcal{BC}$) ne contribuait pas positivement : la configuration à 7 paramètres (termes de bande forcés à zéro) surpassait la configuration à 9 paramètres dans les tests croisés. La configuration finale retenue (`createAblated("bandcoverage")`) exclut donc ce terme — un résultat contre-intuitif au vu de l'importance du mécanisme de bandes dans les règles du jeu, mais confirmé statistiquement.

4.2 Exploration d'une Évaluation par Réseau de Neurones : BandD-PER

Bien que l'heuristique manuelle soit performante, les contraintes spatiales fortement non linéaires d'Escampe se prêtent bien à l'apprentissage automatique. Le modèle **BandD-**

PER (*Band-Aware Dual-Perspective Equivariant ResNet*) a été conçu comme fonction d'évaluation alternative ou support à l'ordonnancement des coups.

Choix du type de modèle

Modèle	Notes	Recommandation
MLP	Rapide, mais aucune conscience spatiale ni raisonnement relationnel	À éviter
CNN	Exploite les motifs spatiaux locaux et l'équivariance par translation	Acceptable, légèrement surdimensionné
NNUE	Mises à jour incrémentales rapides, conception dual-perspective, activations ClippedReLU	Trop conçu pour la taille du problème, mais le dual-perspective et le ClippedReLU sont empruntés
ResNet	Bon si une évaluation de base existe, sans conscience spatiale intrinsèque	Retenu
GNN	Invariant aux symétries, mais graphe dynamique, complexe et lent	À éviter

TABLE 1 – Comparaison des architectures candidates pour la fonction d'évaluation.

La combinaison retenue est un **ResNet dual-perspective avec encodeur CNN spatial**, inspirée du design NNUE [12] (dual-perspective, ClippedReLU, raccourci de sortie direct) et des réseaux siamois [13].

Architecture

BandDPER repose sur six composants principaux : (i) un encodeur CNN spatial siamois à poids partagés, (ii) des canaux d'entrée encodant la topologie des bandes et les contraintes légales, (iii) une carte d'attaquants relative à la licorne (inspirée de l'encodage HalfKP de NNUE), (iv) un tronc résiduel avec ClippedReLU, (v) une injection scalaire des comptes d'échappatoire, et (vi) un raccourci de sortie direct pour le signal de passe forcé.

Encodeur spatial partagé (SharedSpatialEncoder). Chaque perspective est encodée séparément via le même réseau (poids partagés, principe des réseaux siamois [13]), sous la forme d'un tenseur $[B, 16, 6, 6]$ représentant 16 canaux de caractéristiques :

Canal	Contenu	Type
0	Positions de mes paladins	Dynamique 0/1
1	Position de ma licorne	Dynamique 0/1
2	Positions des paladins adverses	Dynamique 0/1
3	Position de la licorne adverse	Dynamique 0/1
4	Masque des cases à bande 1	Fixe 0/1
5	Masque des cases à bande 2	Fixe 0/1
6	Masque des cases à bande 3	Fixe 0/1
7	Masque de contrainte de départ	Dynamique 0/1
8–10	Cases d'atterrissage légales (bandes 1, 2, 3)	Dynamique 0/1
11–14	Taux d'occupation par rangée/-colonne (moi / adv.)	Dynamique [0, 1]
15	Carte d'attaquants relat. à la licorne	Dynamique 0/1

TABLE 2 – Canaux d'entrée pour chaque tenseur de perspective $[B, 16, 6, 6]$.

Le canal 15 place les paladins adverses dans un référentiel centré sur notre licorne (ancrage en (2,2)), inspiré de l'encodage HalfKP de NNUE [12]. Les canaux 8–10 rendent le mécanisme de forçage de tempo explicitement visible. Les canaux 4–6 (masques de bandes) sont fixes et encodent la topologie non-uniforme du plateau.

L'encodeur projette chaque tenseur en un vecteur de dimension 128 via : $\text{Conv2d}(16 \rightarrow 32, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv2d}(32 \rightarrow 64, 3 \times 3, \text{dil.} = 2) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Linear}(2304 \rightarrow 128) \rightarrow \text{BN} \rightarrow \text{ClippedReLU}[0, 1]$. La dilatation 2 élargit le champ réceptif à 5×5 , capturant les motifs de mouvement à moyenne portée.

Fusion et tronc résiduel. Les deux embeddings (256 dimensions) sont concaténés avec deux scalaires normalisés — le nombre de coups légaux de chaque licorne (échappatoire) — formant un vecteur de 258 dimensions. Ce vecteur traverse 3 **ResBlocks** de même dimension ($\text{Linear} \rightarrow \text{Dropout}(0.2) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow \text{BN}$, puis connexion résiduelle + ClippedReLU). Les 3 niveaux correspondent aux trois abstractions naturelles du jeu : proximité/licorne-relative, interaction de bandes/tempo, forçage/échappatoire.

Tête de sortie et raccourci. $\text{Linear}(258 \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64 \rightarrow 1) \rightarrow \tanh$, produisant un scalaire dans $[-1, +1]$. Un paramètre appris unique (**w_forced_pass**) s'additionne directement à la sortie brute sans traverser le tronc, inspiré du raccourci PSQT de Stockfish HalfKAv2 [12]. Ce bypass garantit un gradient fort pour les positions de passe forcé ($\sim 2\text{--}5\%$ du corpus), sans surcharger le tronc.

Composant	Paramètres
Encodeur partagé (utilisé $\times 2$, poids identiques)	$\sim 312\,000$
Tronc résiduel (3 ResBlocks, dim=258)	$\sim 402\,648$
Tête de sortie	$\sim 16\,577$
Total	$\sim 730\,625$

TABLE 3 – Compte des paramètres par composant ($\approx 2,9$ Mo en float32).

Bilan paramétrique. Une variante allégée (`embed_dim=64`, 2 ResBlocks) atteint $\sim 185\text{K}$ paramètres pour une intégration plus rapide.

Génération des données et entraînement

Les étiquettes sont générées par **amorçage Minimax** : pour chaque position échantillonnée en auto-jeu, un alpha-bêta à profondeur 4–6 calcule le score normalisé dans $[-1, +1]$. On vise 50K–100K positions labélisées. Les exemples stockent des triplets (`état`, `bande_dernier_coup`, `label`) pour conserver l'information de contrainte de bande et de passe forcé.

L'entraînement utilise une perte MSE, l'optimiseur AdamW ($\text{lr} = 10^{-3}$, weight decay 10^{-4}), un scheduler *CosineAnnealingWarmRestarts* sur 40 époques (batch 256, split 90/10), et un *gradient clipping* à 1.0. Les poids sont initialisés par Xavier uniforme (gain 0,5).

Pipeline d'export et optimisations envisagées

Pour l'intégration Java, BatchNorm est replié analytiquement dans les couches adjacentes avant l'export (éliminant les divisions au moment de l'inférence), puis les poids sont sérialisés en JSON ou exportés via ONNX Runtime (latence cible $\sim 1\text{--}2\ \mu\text{s}$ pour le modèle INT8 de 25K paramètres quantifiés). Une **tête de politique** ajoutant une distribution sur les ~ 96 coups légaux a été envisagée pour améliorer l'ordonnancement des coups à la racine.

Limites et statut d'intégration

BandDPER n'a pas été retenu comme heuristique principale : la latence d'inférence depuis Java reste incompatible avec les contraintes de temps par nœud de l'alpha-bêta profond. Son usage le plus prometteur reste l'ordonnancement à la racine (coût ponctuel, non répété), laissé comme piste de développement futur.

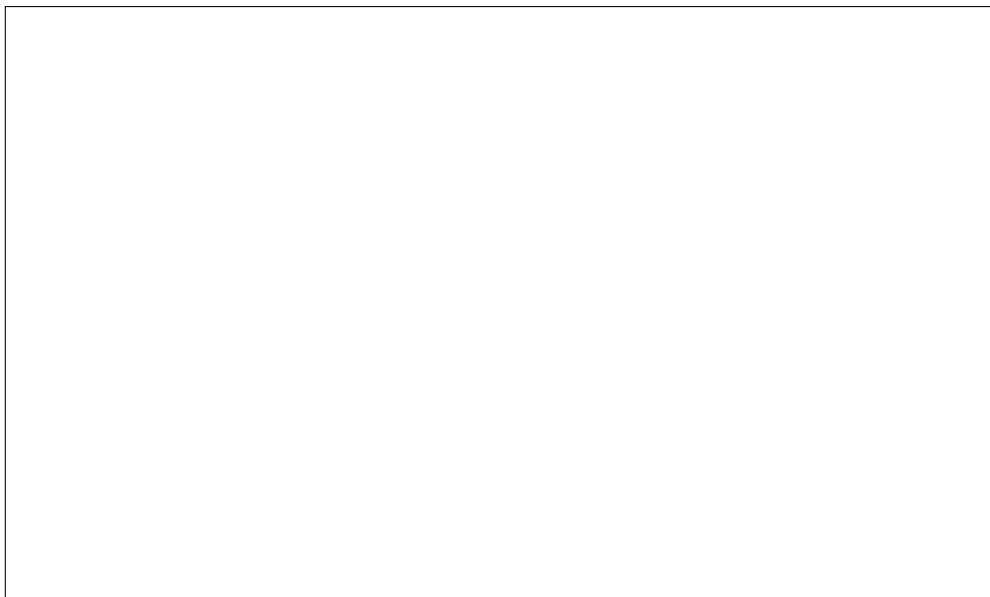


FIGURE 1 – Architecture du réseau BandDPER (encodeur CNN dual-perspective + tronc résiduel) ou courbes de perte MSE entraînement/validation.

5 Stratégie Initiale : Placement et Ouvertures

5.1 Principes de Placement (Opening Book)

L'issue de la partie dépend grandement du positionnement initial. Contrairement aux Échecs, le plateau de départ d'Escampe est défini par les joueurs. Notre agent intègre une bibliothèque de placements de départ (*Opening Book*) garantissant de solides propriétés structurelles :

- **Couverture de toutes les bandes** : Les Paladins doivent couvrir idéalement les bandes 1, 2 et 3. Un déséquilibre offrirait à l'adversaire l'opportunité de cibler une bande où l'agent serait contraint de passer.
- **Dispersion sur les colonnes** : Éviter les grappes de Paladins qui se bloquent mutuellement.
- **Sécurité de la Licorne** : Ne pas murer la Licorne avec ses propres pièces, tout en s'assurant qu'elle ne soit pas au centre géométrique sans couverture, privilégiant les cases "double-bande" pour l'équilibre entre mobilité et défense.

Différentes stratégies (*Balanced*, *Central Pressure*, *Symmetric*...) sont tirées aléatoirement pour varier les parties et éviter la prévisibilité. En dehors de la phase de placement, l'agent met à profit les temps de réponse de l'arbitre grâce au **Pondering** [5] : après avoir joué son coup, il prédit le meilleur coup adverse (100 ms de recherche rapide), applique ce coup hypothétique sur une copie du plateau, puis lance en arrière-plan un thread de recherche profonde (budget 4s). Si l'adversaire joue effectivement le coup prédit (*ponder hit*), la réponse est déjà calculée et jouée instantanément, économisant du temps sur le budget principal. En cas d'échec (*ponder miss*), le thread est interrompu et la recherche repart de zéro normalement.

5.2 Stratégie par Phase de Jeu

Une stratégie différenciée par phase améliore significativement les performances :

Début de partie Placement équilibré couvrant plusieurs bandes (1, 2, 3). Licorne protégée derrière des Paladins sans être enfermée. Éviter les couloirs directs vers sa Licorne.

Milieu de partie Jouer le tempo : choisir les cases d'arrivée pour imposer une mauvaise bande adverse ou forcer un passe. Créer des menaces doubles sur la Licorne tout en maintenant une défense active. Limiter les coups qui offrent une bande favorable à l'adversaire.

Fin de partie Chercher les séquences forcantes (capture en 1-3 coups) via une recherche plus profonde grâce à l'approfondissement itératif et à la réduction du budget temporel. En défense, maximiser la mobilité de la Licorne et casser les lignes d'attaque.

6 Gestion du Temps Réel

La communication avec le serveur impose des temps limites stricts sous peine de défaite immédiate. Nous avons implémenté un **TimeManager** [4] hybride avec gestion dynamique des bornes.

- **Estimation du temps par coup** : Le temps de base alloué est calculé en divisant le temps global restant par une estimation dynamique du nombre de coups restants. Cette estimation s'adapte selon la réserve : 40 coups supposés s'il reste beaucoup de temps ($>200s$), puis descend par paliers (25, 15) jusqu'à 8 coups en fin de partie critique.
- **Facteur de Complexité** : Un multiplicateur ajuste le budget temporel en fonction du nombre de coups légaux. Si le joueur n'a qu'un ou deux coups quasi-forcés (contrainte de bande), le budget est drastiquement réduit (facteur de 0.3) pour économiser du temps. Dans une position très ouverte (≥ 20 coups), le budget augmente (facteur 1.5).
- **Soft Bound / Hard Bound** :
 - La *Soft Bound* (limite logicielle) correspond au budget temporel de base ajusté. Une fois franchie, la boucle d'*Iterative Deepening* s'interrompt pour renvoyer le meilleur coup de la dernière profondeur complète.
 - La *Hard Bound* (limite matérielle) agit comme une sécurité absolue, fixée à $3 \times$ la *Soft Bound* (avec un plafond strict à un cinquième du temps total restant). Si elle est atteinte en plein milieu de l'évaluation d'un nœud (lors des appels récursifs du Minimax), l'algorithme coupe instantanément la branche et retourne l'évaluation heuristique courante, empêchant ainsi le dépassement de temps fatal.

7 Analyse des Performances et Tests

Les performances de l'IA ont été évaluées via un moteur de test autonome opposant plusieurs versions de notre agent.

7.1 Évaluation Elo et Test SPRT

L'amélioration des heuristiques et l'optimisation des structures de données ont été quantifiées via un système de tournoi automatique (**TournamentRunner**) opposant de multiples configurations de notre agent. Le classement repose sur le rating Elo avec mise à jour $R' = R + K(S - E)$, où $E = 1/(1 + 10^{(R_B - R_A)/400})$ et $K = 32$. Les configurations testées incluent différentes profondeurs (2 à 6), les deux algorithmes de recherche (AlphaBetaKH et NegamaxKH), les trois variants d'heuristique (Default, SimplexFull, BandCoverage-ablated) et l'activation ou non du Pondering.

Pour valider statistiquement qu'une modification apporte un gain Elo réel avant de la valider, nous utilisons le **SPRT** [11] (*Sequential Probability Ratio Test*). Le SPRT compare deux hypothèses : H_0 (les deux agents sont équivalents, $Elo = 0$) contre H_1 (l'un est meilleur, $Elo > 0$). Le test s'arrête dès que les données accumulées sont suffisantes pour accepter ou rejeter H_0 au seuil de confiance fixé (95%), évitant de jouer des parties inutiles. Ce processus itératif, inspiré des pratiques de Stockfish, garantit que seules les améliorations réelles progressent vers la version finale.

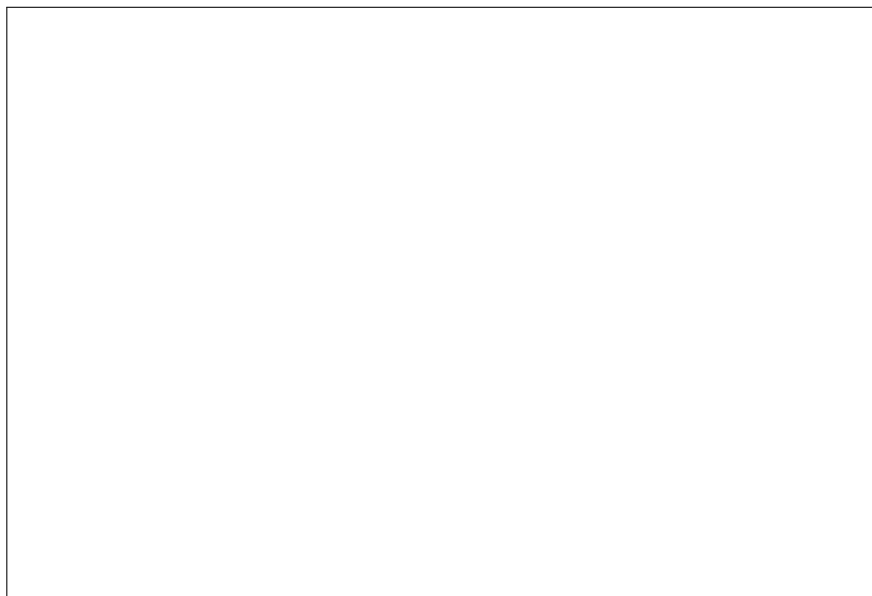


FIGURE 2 – Évolution du classement Elo des différentes configurations testées (tests SPRT).

7.2 Validation de la Conduite Autonome

Les protocoles de communication avec l'arbitre ont été validés localement en s'assurant qu'aucun dépassement de délai n'intervenait et que tous les coups générés restaient strictement conformes aux règles topologiques du jeu.

8 Conclusion et Difficultés Rencontrées

La mise au point de cette IA pour le jeu d'Escampe a été un défi algorithmique enrichissant. Les principales difficultés ont résidé dans :

- La compréhension initiale des symétries brisées du plateau (l'absence d'uniformité due aux bandes asymétriques complexifie fortement la pré-détermination de patterns

géométriques réguliers).

- Le débogage subtil inhérent à l'algorithme Alpha-Bêta, notamment lors de l'intégration de l'ordonnancement dynamique des coups et de la gestion asynchrone des limites temporelles (*Hard bounds*).
- La conception d'une heuristique qui ne se laisse pas piéger par l'effet d'horizon, exacerbé dans Escampe par les réactions en chaîne de la mécanique de passe obligatoire.

L'agent final représente une synthèse robuste entre techniques classiques de la programmation de jeux à information parfaite et optimisation mathématique de topologie.

Références

- [1] Chess Programming Wiki, *Alpha-Beta*, <https://www.chessprogramming.org/Alpha-Beta>
- [2] Chess Programming Wiki, *Bitboards*, <https://www.chessprogramming.org/Bitboards>
- [3] Chess Programming Wiki, *Iterative Deepening*, https://www.chessprogramming.org/Iterative_Deepening
- [4] Chess Programming Wiki, *Time Management*, https://www.chessprogramming.org/Time_Management
- [5] Chess Programming Wiki, *Pondering*, <https://www.chessprogramming.org/Pondering>
- [6] Chess Programming Wiki, *Move Ordering*, https://www.chessprogramming.org/Move_Ordering
- [7] Chess Programming Wiki, *Killer Heuristic*, https://www.chessprogramming.org/Killer_Heuristic
- [8] Chess Programming Wiki, *Null Move Pruning*, https://www.chessprogramming.org/Null_Move_Pruning
- [9] Chess Programming Wiki, *Late Move Reductions*, https://www.chessprogramming.org/Late_Move_Reductions
- [10] Chess Programming Wiki, *Zobrist Hashing*, https://www.chessprogramming.org/Zobrist_Hashing
- [11] Chess Programming Wiki, *Match Statistics (SPRT)*, https://www.chessprogramming.org/Match_Statistics
- [12] Yu Nasu, *Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi*, The 28th World Computer Shogi Championship Appeal Document, 2018. <https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html>
- [13] J. Bromley et al., *Signature Verification using a Siamese Time Delay Neural Network*, NeurIPS, 1993. <https://papers.nips.cc/paper/1993/hash/0d4262ad58b4d3866d5aa0f4f9e1c06b-Abstract.html>
- [14] D. Silver et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, arXiv :1712.01815, 2017. <https://arxiv.org/abs/1712.01815>